MODUL 03 · BAB II · ANALISIS RUNNING TIME

Konsep Running Time dan Model Analisis Algoritma

Mengukur kecepatan algoritma tanpa stopwatch: menghitung biaya dan banyaknya eksekusi setiap langkah.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan arti running time $T(n)$ dan hubungannya dengan ukuran masukan n.
- Menghitung running time dengan metode *cost* × jumlah operasi, termasuk aturan $n+1$ untuk kepala perulangan.
- Membedakan kasus terbaik, terburuk, dan rata-rata serta alasan analisis biasanya berfokus pada kasus terburuk.
- Menyederhanakan fungsi running time menjadi bentuk $an+b$ atau an^2+bn+c dan mengenali orde pertumbuhannya.

PRASYARAT

- Modul 01: pseudocode dan struktur perulangan.
- Modul 02 (disarankan): penelusuran algoritma.

BAB / MATERI ASAL

Materi 2. ALGORITMA INSERTION SORT dan RUNNING TIME (bagian Analisis Running Time dan contoh CARI_BIL_K) PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Kompleksitas Algoritma)

POSISI DALAM URUTAN BELAJAR

Konsep umum running time. Diterapkan secara khusus pada Insertion Sort (Modul 04) dan menjadi dasar Modul 05–07.

DAFTAR ISI

1. Mengapa Tidak Cukup dengan Stopwatch? 1
2. Ukuran Masukan dan Running Time 2
3. Kasus Terbaik, Terburuk, dan Rata-rata 3
4. Menghitung Running Time: Tiga Contoh 3
5. Dari Rumus Lengkap ke Orde Pertumbuhan 6
6. Kesimpulan 7
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 10
- Referensi 10

PENDAHULUAN

1. Mengapa Tidak Cukup dengan Stopwatch?

Cara paling langsung untuk mengetahui kecepatan sebuah algoritma adalah menjalankannya lalu mengukur waktunya. Namun hasil pengukuran itu bergantung pada banyak hal di luar algoritma: kecepatan prosesor, bahasa pemrograman, kompiler, program lain yang sedang berjalan, bahkan suhu

ruangan yang memengaruhi prosesor. Algoritma yang sama bisa berjalan 0,2 detik di satu laptop dan 2 detik di laptop lain.

Yang kita butuhkan adalah ukuran yang mencerminkan **sifat algoritmanya sendiri**, bukan sifat mesinnya. Ukuran itu adalah **running time**: banyaknya langkah dasar yang dijalankan algoritma, dinyatakan sebagai fungsi dari ukuran masukan. Dengan ukuran ini kita dapat menjawab pertanyaan yang jauh lebih berguna: "Jika datanya menjadi sepuluh kali lebih banyak, berapa kali lebih lama algoritma ini berjalan?"

Modul ini membahas konsep running time secara umum: apa yang dihitung, bagaimana menghitungnya, dan bagaimana membedakan kasus terbaik dan terburuk. Penerapan lengkapnya pada algoritma pengurutan dibahas di Modul 04 (Insertion Sort).

PEMBAHASAN INTI

2. Ukuran Masukan dan Running Time

2.1 Ukuran masukan

Ukuran masukan, dilambangkan n, adalah besaran yang paling menentukan banyaknya pekerjaan algoritma. Pemilihannya bergantung pada masalah:

- Pengurutan dan pencarian pada array: n = banyaknya elemen array.
- Perkalian dua matriks: ukuran matriksnya (banyaknya baris dan kolom).
- Algoritma graf: dua besaran sekaligus, yaitu banyaknya simpul $|V|$ dan banyaknya busur $|E|$ (Modul 16–24).

2.2 Running time

DEFINISI

Running time suatu algoritma untuk masukan tertentu adalah banyaknya operasi dasar (langkah) yang dijalankan. Karena bergantung pada ukuran masukan, running time ditulis sebagai fungsi $T(n)$.

Untuk menghitungnya kita memakai model sederhana yang juga dipakai Cormen dkk. (2009): setiap baris pseudocode membutuhkan waktu yang **konstan** setiap kali dijalankan. Baris ke-i membutuhkan waktu c_i, yang disebut **cost**. Nilai c_i bisa berbeda untuk baris yang berbeda, tetapi untuk baris yang sama nilainya selalu sama. Kita tidak perlu tahu berapa nilai c_i sebenarnya, karena justru itulah bagian yang bergantung pada mesin.

2.3 Metode cost × jumlah operasi

Running time diperoleh dengan menjumlahkan, untuk setiap baris, hasil kali **cost** baris itu dengan **jumlah operasi**, yaitu berapa kali baris tersebut dijalankan:

$$T(n) = \sum_{\text{setiap baris } i} c_i \times (\text{jumlah operasi baris } i).$$

Langkah kerjanya:

1. Beri nomor setiap baris dan tetapkan cost $c_1, c_2, \ldots$

2. Untuk setiap baris, hitung berapa kali baris itu dijalankan sebagai fungsi dari n.
3. Kalikan dan jumlahkan.
4. Kelompokkan suku-suku berdasarkan pangkat n, lalu ganti gabungan konstanta dengan huruf baru (a, b, c, ...).

2.4 Aturan penting: kepala perulangan dijalankan satu kali lebih banyak

Perhatikan perulangan `for i = 1 to n`. Isi perulangan dijalankan untuk $i = 1, 2, \ldots, n$, yaitu n kali. Tetapi baris `for` itu sendiri, yang memeriksa apakah perulangan masih berlanjut, dijalankan $n + 1$ kali: n kali menghasilkan "lanjut", ditambah satu kali lagi untuk $i = n + 1$ yang menghasilkan "berhenti".

INTUISI

Untuk perulangan `for j = 2 to n`, isi perulangan dijalankan $n - 1$ kali dan kepala perulangan dijalankan $(n - 1) + 1 = n$ kali. Aturannya selalu: **kepala = isi + 1**. Hal yang sama berlaku untuk `while`: kondisi diperiksa sekali lagi ketika bernilai salah.

PEMBAHASAN INTI

3. Kasus Terbaik, Terburuk, dan Rata-rata

Untuk ukuran masukan yang sama, running time bisa berbeda tergantung **isi** masukannya. Mencari bilangan 5 di array berukuran 1.000 bisa selesai di elemen pertama atau baru selesai di elemen terakhir. Karena itu kita membedakan:

Kasus	Arti	Contoh pada pencarian linear
Terbaik (*best case*)	Running time paling kecil di antara semua masukan berukuran n	Bilangan yang dicari ada di elemen pertama
Terburuk (*worst case*)	Running time paling besar di antara semua masukan berukuran n	Bilangan yang dicari tidak ada di array
Rata-rata (*average case*)	Nilai harapan running time, dengan asumsi distribusi masukan tertentu	Bilangan berada di posisi acak, rata-rata sekitar separuh array diperiksa

Tabel 1. Tiga jenis analisis running time.

Analisis biasanya berfokus pada **kasus terburuk**, dengan tiga alasan (Cormen dkk., 2009):

1. Kasus terburuk adalah **jaminan**: algoritma tidak akan pernah lebih lambat dari itu, apa pun masukannya.
2. Untuk banyak algoritma, kasus terburuk **sering terjadi**. Contohnya, mencari data yang memang tidak ada di basis data.
3. Kasus rata-rata **sering sama buruknya** dengan kasus terburuk dalam orde pertumbuhan, dan menghitungnya membutuhkan asumsi tentang distribusi masukan yang belum tentu realistis.

CONTOH DAN PENERAPAN

4. Menghitung Running Time: Tiga Contoh

4.1 Contoh 1: menjumlahkan n bilangan

JUMLAH_n_BIL(A)

```
1  jumlah = 0
2  for i = 1 to n
3      jumlah = jumlah + A[i]
4  return jumlah
```

Baris	Cost	Jumlah operasi
1	c_1	1
2	c_2	$n+1$
3	c_3	n
4	c_4	1

Tabel 2. Cost dan jumlah operasi JUMLAH_n_BIL.

$$T(n) = c_1 + c_2(n+1) + c_3 n + c_4 = (c_2 + c_3)\, n + (c_1 + c_2 + c_4).$$

Dengan $a = c_2 + c_3$ dan $b = c_1 + c_2 + c_4$, diperoleh $T(n) = an + b$: **fungsi linier**. Di sini tidak ada kasus terbaik atau terburuk yang berbeda, karena semua elemen selalu dijumlahkan.

4.2 Contoh 2: mencari bilangan k (dari bahan kuliah)

Soal: buat algoritma untuk menyelidiki apakah array A berisi n bilangan sembarang memuat bilangan k. Jika ya, kembalikan indeksnya; jika tidak, kembalikan -1. Tentukan running time kasus terburuk.

CARI_BIL_K(A, k)

```
1  for i = 1 to A.length
2      if (k == A[i])
3          then return i
4  return -1
```

Kasus terburuk terjadi bila k tidak ada di A. Semua elemen diperiksa, baris 3 tidak pernah dijalankan, dan algoritma berakhir di baris 4.

Baris	Cost	Terburuk (k tidak ada)	Terbaik ($k = A[1]$)
1	c_1	$n+1$	1
2	c_2	n	1
3	c_3	0	1
4	c_4	1	0

Tabel 3. Running time CARI_BIL_K pada kasus terburuk dan kasus terbaik.

Kasus terburuk:

$$T(n) = c_1(n+1) + c_2 n + c_3 \cdot 0 + c_4 \cdot 1 = (c_1 + c_2)\, n + (c_1 + c_4) = an + b.$$

Kasus terbaik: $T(n) = c_1 + c_2 + c_3$, yaitu **konstanta** yang tidak bergantung pada n. Perhatikan bagaimana satu algoritma dapat memiliki running time konstan pada kasus terbaik tetapi linier pada kasus terburuk.

MENGAPA BARIS 1 PADA KASUS TERBAIK HANYA DIHITUNG 1 KALI?

Ketika $k = A[1]$, algoritma keluar lewat `return` di baris 3 pada putaran pertama. Pemeriksaan kepala perulangan yang "ke-$n+1$" tidak pernah terjadi karena fungsi sudah selesai. Aturan "kepala = isi + 1" hanya berlaku bila perulangan berakhir secara normal.

4.3 Contoh 3: perulangan bersarang dan deret aritmetika

Algoritma berikut menghitung banyaknya pasangan (i, j) dengan $i < j$ yang nilainya sama, misalnya untuk mendeteksi data ganda.

HITUNG_PASANGAN_SAMA(A)

```
1 hitung = 0
2 for i = 1 to n - 1
3     for j = i + 1 to n
4         if A[i] == A[j]
5             hitung = hitung + 1
6 return hitung
```

Untuk setiap i, perulangan dalam menjalankan isinya sebanyak $n - i$ kali. Total eksekusi baris 4 adalah

$$\sum_{i=1}^{n-1} (n - i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

Hasil ini memakai rumus **deret aritmetika** (deret hitung) yang akan sering muncul:

$$1 + 2 + 3 + \cdots + m = \sum_{j=1}^{m} j = \frac{m(m+1)}{2}.$$

Baris 3 (kepala perulangan dalam) dijalankan $(n - i) + 1$ kali untuk setiap i, sehingga totalnya $\frac{n(n-1)}{2} + (n-1)$. Baris 2 dijalankan n kali, baris 1 dan 6 masing-masing sekali, dan baris 5 dijalankan antara 0 kali (semua data berbeda) sampai $\frac{n(n-1)}{2}$ kali (semua data sama). Kasus terburuknya:

$$T(n) = c_1 + c_2 n + c_3 \left(\tfrac{n(n-1)}{2} + n - 1\right) + (c_4 + c_5)\tfrac{n(n-1)}{2} + c_6 = an^2 + bn + c,$$

dengan $a = \frac{c_3 + c_4 + c_5}{2}$. Hasilnya **fungsi kuadrat**. Bentuk seperti ini akan kita temui lagi pada kasus terburuk Insertion Sort di Modul 04.

4.4 Memeriksa hitungan dengan program

Kita dapat memeriksa hitungan jumlah operasi dengan menambahkan penghitung ke dalam program. Yang dihitung adalah banyaknya eksekusi baris perbandingan, bukan waktu jam, sehingga hasilnya sama di komputer mana pun.

Python: menghitung jumlah operasi baris perbandingan

```python
def cari_bil_k(A, k):
    cek = 0
    for i in range(len(A)):
        cek += 1                    # satu kali eksekusi baris "if (k == A[i])"
        if k == A[i]:
            return i + 1, cek       # indeks gaya pseudocode (mulai dari 1)
    return -1, cek

def hitung_pasangan_sama(A):
    n, cek, hitung = len(A), 0, 0
    for i in range(n - 1):
        for j in range(i + 1, n):
            cek += 1
            if A[i] == A[j]:
                hitung += 1
    return hitung, cek

for n in (10, 100, 1000):
    A = list(range(n))
    print(n, cari_bil_k(A, -5), hitung_pasangan_sama(A), n * (n - 1) // 2)
```

Keluaran program

```
10 (-1, 10) (0, 45) 45
100 (-1, 100) (0, 4950) 4950
1000 (-1, 1000) (0, 499500) 499500
```

Ketika n naik 10 kali, jumlah perbandingan `CARI_BIL_K` juga naik 10 kali (linier), sedangkan `HITUNG_PASANGAN_SAMA` naik sekitar 100 kali (kuadrat), tepat sesuai rumus $\frac{n(n-1)}{2}$.

ANALISIS DAN HUBUNGAN KONSEP

5. Dari Rumus Lengkap ke Orde Pertumbuhan

Rumus running time yang lengkap memuat banyak konstanta yang nilainya bergantung pada mesin. Untuk membandingkan algoritma, yang paling penting adalah **suku dengan pangkat tertinggi**, karena suku itulah yang mendominasi ketika n besar. Contohnya $T(n) = 3n^2 + 100n + 500$:

n	$3n^2$	$100n$	500	**Porsi suku $3n^2$**
10	300	1.000	500	16,7%
100	30.000	10.000	500	74,1%
1.000	3.000.000	100.000	500	96,8%
10.000	300.000.000	1.000.000	500	99,7%

Tabel 4. Kontribusi tiap suku pada $T(n) = 3n^2 + 100n + 500$.

Untuk n besar, suku $100n$ dan 500 hampir tidak berarti, dan koefisien 3 hanya menggeser nilai dengan faktor tetap. Karena itu kita cukup mengatakan algoritma ini memiliki **orde pertumbuhan kuadrat**, atau, dalam notasi yang dibahas di Modul 06, $T(n) = \Theta(n^2)$.

Bahan kuliah memberikan daftar bentuk orde pertumbuhan yang umum, dari yang paling efisien: konstan $O(1)$ (misalnya mengakses elemen array dengan indeks tertentu), logaritmik $O(\lg n)$ (Binary Search, Modul 02), linier $O(n)$ (menelusuri seluruh array), $O(n \lg n)$ (Merge Sort, Modul 08), kuadrat $O(n^2)$ (Insertion Sort, Modul 04), polinomial $O(n^k)$, eksponensial $O(2^n)$, dan faktorial $O(n!)$. Seberapa besar perbedaan antarkelas ini dibahas secara kuantitatif di Modul 05.

Hubungan dengan modul lain. Modul 04 menerapkan metode cost × jumlah operasi pada Insertion Sort dengan perulangan `while` yang jumlah putarannya bergantung pada isi data, sehingga kasus terbaik dan terburuknya berbeda bentuk (linier vs kuadrat). Modul 05 membahas mengapa perbedaan bentuk itu sangat penting, dan Modul 06–07 memformalkan "cukup ambil suku dominan" menjadi notasi asimtotik.

KESIMPULAN

6. Kesimpulan

- Running time $T(n)$ adalah banyaknya langkah dasar yang dijalankan algoritma sebagai fungsi ukuran masukan n. Ukuran ini tidak bergantung pada mesin.
- Metode cost × jumlah operasi: setiap baris i berbiaya konstan c_i; running time adalah jumlah dari c_i dikali banyaknya eksekusi baris i.
- Kepala perulangan yang berakhir normal dijalankan satu kali lebih banyak daripada isinya.
- Kasus terbaik, terburuk, dan rata-rata dapat berbeda untuk ukuran masukan yang sama. Kasus terburuk paling sering dipakai karena memberikan jaminan.
- Setelah disederhanakan, running time biasanya berbentuk seperti $an + b$ (linier) atau $an^2 + bn + c$ (kuadrat). Suku berpangkat tertinggi menentukan orde pertumbuhan.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Mengapa running time dinyatakan dengan konstanta c_i dan bukan dalam detik?

A. Karena konstanta lebih mudah dihitung daripada detik.
B. Agar analisis menggambarkan sifat algoritma dan tidak bergantung pada mesin, bahasa, atau kompiler.
C. Karena semua baris pseudocode membutuhkan waktu yang persis sama.
D. Karena komputer tidak dapat mengukur waktu secara akurat.

2 PERHITUNGAN

Sebuah perulangan ditulis `for j = 3 to n` dengan $n \geq 3$ dan berakhir normal. Berapa kali kepala perulangan dijalankan, dan berapa kali isinya dijalankan?

3 PERHITUNGAN

Tentukan jumlah operasi setiap baris dan running time $T(n)$ algoritma berikut, lalu sederhanakan menjadi bentuk umum.

RATA_RATA(A)

```
1  total = 0
2  for i = 1 to n
3      total = total + A[i]
4  rata = total / n
5  return rata
```

4 BENAR/SALAH DENGAN ALASAN

"Kasus terbaik `CARI_BIL_K` terjadi ketika array berukuran kecil." Benar atau salah?

5 PERHITUNGAN

Berapa kali baris `x = x + 1` dijalankan oleh potongan berikut sebagai fungsi n? Apa orde pertumbuhannya?

Potongan algoritma

```
for i = 1 to n
    for j = 1 to i
        x = x + 1
```

6 ANALISIS

Algoritma P memiliki running time $T_P(n) = 50n + 1000$ dan algoritma Q memiliki $T_Q(n) = 2n^2$ (satuan langkah). (a) Untuk $n = 10$, mana yang lebih cepat? (b) Untuk $n = 100$? (c) Mulai nilai n berapa P selalu lebih cepat?

7 **PILIHAN GANDA**

Alasan yang **bukan** merupakan alasan utama analisis berfokus pada kasus terburuk adalah ...

A. Kasus terburuk memberikan batas atas yang dijamin untuk semua masukan.
B. Kasus terburuk sering terjadi dalam praktik.
C. Kasus terburuk selalu jauh lebih mudah dihitung daripada kasus terbaik.
D. Kasus rata-rata sering memiliki orde yang sama dengan kasus terburuk.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Nilai c_i mewakili biaya yang bergantung pada mesin. Dengan menjaganya sebagai simbol, kita fokus pada berapa kali setiap baris dijalankan, yang merupakan sifat algoritma. Pilihan C keliru: baris yang berbeda boleh memiliki cost berbeda.

Soal 2

Isi perulangan dijalankan untuk $j = 3, 4, \ldots, n$, yaitu $n - 2$ **kali**. Kepala perulangan dijalankan satu kali lebih banyak, yaitu $n - 1$ **kali** (termasuk pemeriksaan $j = n + 1$ yang menghentikan perulangan).

Soal 3

Jumlah operasi: baris 1 → 1, baris 2 → $n + 1$, baris 3 → n, baris 4 → 1, baris 5 → 1.

$T(n) = c_1 + c_2(n+1) + c_3 n + c_4 + c_5 = (c_2 + c_3)n + (c_1 + c_2 + c_4 + c_5) = an + b$, fungsi **linier**.

Soal 4

Salah. Kasus terbaik dan terburuk dibandingkan untuk ukuran masukan n yang **sama**. Kasus terbaik `CARI_BIL_K` adalah ketika k berada di $A[1]$, sehingga algoritma selesai setelah satu perbandingan, berapa pun besarnya n.

Soal 5

Untuk i tertentu, baris itu dijalankan i kali. Totalnya $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ kali. Karena suku dominannya $\frac{n^2}{2}$, orde pertumbuhannya **kuadrat**.

Soal 6

(a) $n = 10$: $T_P = 1500$, $T_Q = 200$, **Q lebih cepat**. (b) $n = 100$: $T_P = 6000$, $T_Q = 20000$, **P lebih cepat**. (c) P lebih cepat bila $50n + 1000 < 2n^2$, yaitu $n^2 - 25n - 500 > 0$. Akar positifnya $n = \frac{25 + \sqrt{625 + 2000}}{2} \approx 38{,}1$. Jadi untuk $n \geq 39$ P selalu lebih cepat. (Cek: $n = 38$ memberi 2900 vs 2888, Q masih sedikit lebih cepat; $n = 39$ memberi 2950 vs 3042, P lebih cepat.) Ini menggambarkan bahwa untuk n besar, orde pertumbuhan lebih menentukan daripada konstanta.

Soal 7

Jawaban: C. Kasus terburuk tidak selalu lebih mudah dihitung. Alasan sebenarnya adalah A, B, dan D, sebagaimana dijelaskan Cormen dkk. (2009).

Referensi

Sumber utama: Materi 2. ALGORITMA INSERTION SORT dan RUNNING TIME (bagian Analisis Running Time dan contoh CARI_BIL_K); PENGERTIAN ALGORITMA PEMROGRAMAN (bagian Kompleksitas Algoritma) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.

2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)